MediConnect

Hospital Management Backend Prototype

ITEC 116 - Systems Integration and Architecture 2

Prepared by: Raffy Jay Ajero
Documentation date: 19 October 2026

Source repository: github.com/Ajerojay/MediConnect
Video recording: To be supplied by the student after recording the required demonstration.

1. Project Overview

MediConnect organizes patient profiles, doctor appointments and consultation records. Patients request appointments, doctors review bookings and document visits, and administrators manage accounts and directories. The prototype provides a working REST backend and an optional web portal.

2. System Analysis

The system addresses disconnected patient information and manual appointment tracking. Its objectives are to store related records consistently, protect each role's operations, prevent conflicting active bookings, and make CRUD behavior verifiable in Postman.

User	Main operations
Patient	Register, sign in, book or cancel visits, read own medical records.
Doctor	Review assigned appointments, approve visits, maintain consultation records.
Administrator	Maintain patient/doctor profiles, inspect accounts, enable or disable access.

The scope excludes billing, admissions, laboratory processing and insurance. Only synthetic information is used for verification. This is an academic prototype; production healthcare operation requires additional operational and privacy controls.

Contents

1-2. Overview and analysis - page 1
3. Architecture and implementation - page 2
4. Database design - page 3
5. Data dictionary - pages 4-5
6. REST API design - page 6
7-8. Authentication and verification - page 7
9-11. Conclusion, references and submission notes - page 8

3. Architecture and Implementation

The backend is a NestJS modular application using TypeORM repositories and a MySQL database. A Next.js portal consumes the same REST API as Postman. There is no direct browser-to-database access.

Layer	Responsibility	Implementation
Client	Send requests and display results	Postman; Next.js portal
Controller / DTO	Receive HTTP input; validate fields	Resource controllers and ZodPipe
Service	Apply roles, ownership and business rules	Resource service classes
Repository	Read and write persisted records	TypeORM repositories
Database	Store tables, keys and relationships	MySQL 8; six entities

Request and response flow

```
Client (Postman) -> Controller -> Service -> Repository -> MySQL
Client (response) <- Controller <- Service <- Repository
```

For POST /appointments, the JWT guard loads the active patient. The controller validates doctorId, startsAt, reason and notes. The service locks the selected doctor row inside a transaction, checks interval overlap and inserts the booking. The saved appointment is returned with HTTP 201.

Modules

Authentication handles password verification and JWT issuance. Patients and Doctors provide profile CRUD. Appointments handles booking and status changes. Medical Records manages consultations and prescriptions. Users provides administrator account controls.

Implementation references

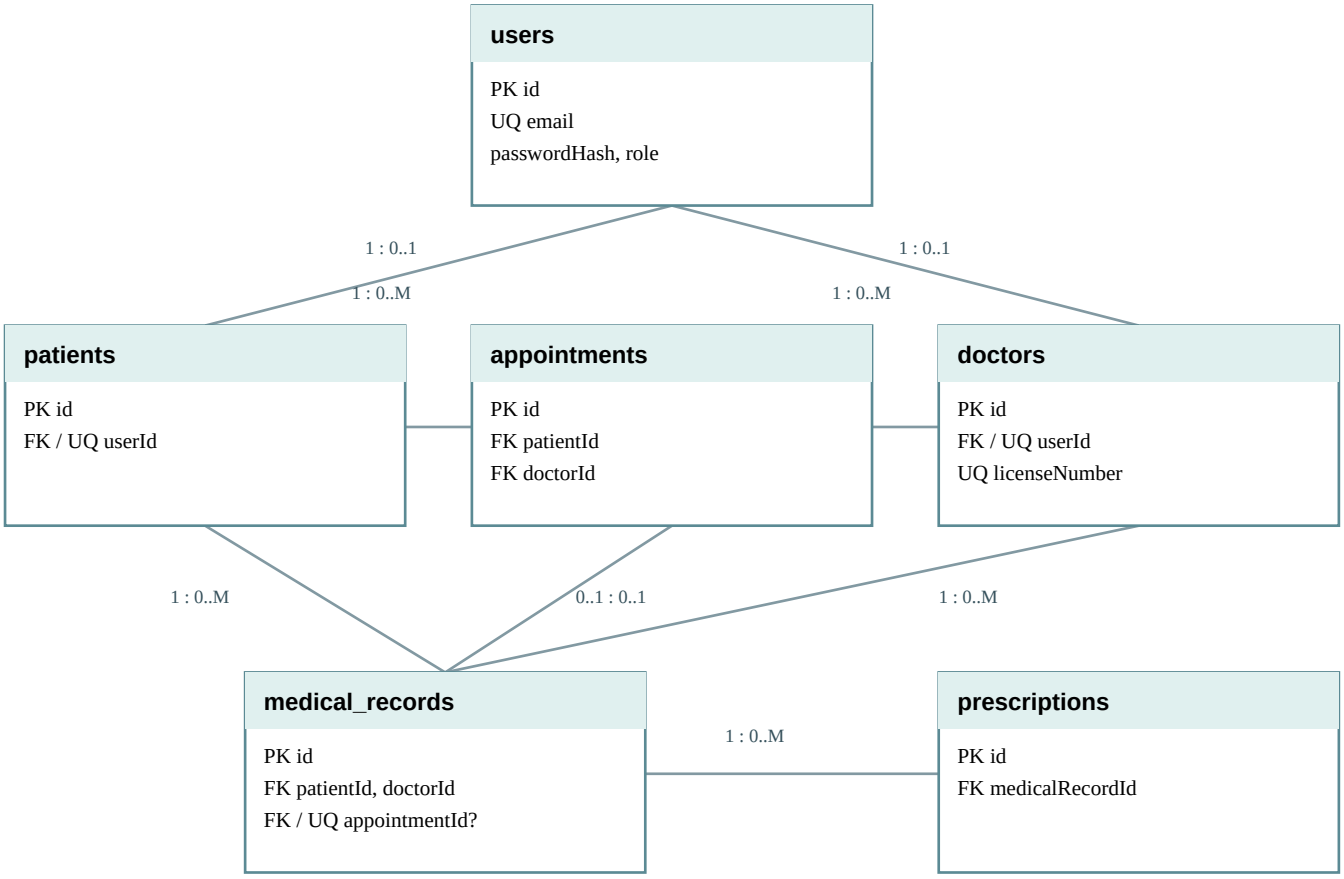
```
backend/src/app.module.ts
backend/src/auth/auth.ts
backend/src/appointments/appointments.ts
backend/src/database/entities.ts
lib/api.ts; actions/; app/(portal)/
```

Startup and persistence

npm run setup explicitly creates the database schema and configured accounts. npm run dev starts the local MySQL instance when managed, the backend and the portal. Normal API startup has synchronization disabled. The development database is separate from other MySQL projects and its files and credentials are excluded from Git.

4. Database Design

Six TypeORM entities define 53 stored columns. PK identifies a row; FK references another table; UQ enforces uniqueness. The diagram below summarizes the keys; the dictionary lists every column.



Relationship	Cardinality / rule
User - Patient / Doctor	One user to zero or one profile; each profile has one user.
Patient / Doctor - Appointment	One-to-many. Each appointment has one patient and one doctor.
Patient / Doctor - MedicalRecord	One-to-many. Clinical records retain their patient and author.
Appointment - MedicalRecord	Optional one-to-one. A record may have no appointment link.
MedicalRecord - Prescription	One-to-many; deleting a record removes its prescriptions.

Foreign keys restrict removal of referenced patients/doctors. The optional appointment reference uses SET NULL. Account/profile data is separated to avoid repeating identity details across appointments. Prescription rows support multiple medications without repeated medication columns.

5. Data Dictionary

Identifiers use VARCHAR(36) UUID values. Datetimes have millisecond precision. Required means NOT NULL. Enum values and defaults match the exported MySQL schema. TypeORM generates identifiers and maintains timestamps.

users

Field	Type / size	Null	Key / default
id	varchar(36)	NO	PK
email	varchar(254)	NO	UQ
passwordHash	varchar(255)	NO	-
fullName	varchar(80)	NO	-
phone	varchar(30)	YES	-
role	ENUM: PATIENT / DOCTOR / ADMIN	NO	default PATIENT
isActive	tinyint	NO	default 1
createdAt	datetime(3)	NO	default CURRENT_TIMESTAMP(3)
updatedAt	datetime(3)	NO	default CURRENT_TIMESTAMP(3)

patients

Field	Type / size	Null	Key / default
id	varchar(36)	NO	PK
userId	varchar(36)	NO	UQ; FK -> users.id
dateOfBirth	date	YES	-
gender	varchar(30)	YES	-
address	varchar(300)	YES	-
emergencyContact	varchar(100)	YES	-
bloodType	varchar(5)	YES	-
allergies	text	YES	-
createdAt	datetime(3)	NO	default CURRENT_TIMESTAMP(3)
updatedAt	datetime(3)	NO	default CURRENT_TIMESTAMP(3)

doctors

Field	Type / size	Null	Key / default
id	varchar(36)	NO	PK
userId	varchar(36)	NO	UQ; FK -> users.id
specialization	varchar(100)	NO	-
licenseNumber	varchar(100)	NO	UQ
createdAt	datetime(3)	NO	default CURRENT_TIMESTAMP(3)
updatedAt	datetime(3)	NO	default CURRENT_TIMESTAMP(3)

5. Data Dictionary

Identifiers use VARCHAR(36) UUID values. Datetimes have millisecond precision. Required means NOT NULL. Enum values and defaults match the exported MySQL schema. TypeORM generates identifiers and maintains timestamps.

appointments

Field	Type / size	Null	Key / default
id	varchar(36)	NO	PK
patientId	varchar(36)	NO	Indexed; FK -> patients.id
doctorId	varchar(36)	NO	Indexed; FK -> doctors.id
startsAt	datetime(3)	NO	-
endsAt	datetime(3)	NO	-
reason	varchar(500)	NO	-
notes	text	YES	-
status	ENUM: PENDING / APPROVED / COMPLETED / CANCELLED	NO	default PENDING
cancellationReason	varchar(500)	YES	-
createdAt	datetime(3)	NO	default CURRENT_TIMESTAMP(3)
updatedAt	datetime(3)	NO	default CURRENT_TIMESTAMP(3)

medical_records

Field	Type / size	Null	Key / default
id	varchar(36)	NO	PK
patientId	varchar(36)	NO	Indexed; FK -> patients.id
doctorId	varchar(36)	NO	Indexed; FK -> doctors.id
appointmentId	varchar(36)	YES	UQ; FK -> appointments.id
diagnosis	text	NO	-
treatment	text	NO	-
consultationNotes	text	NO	-
visitDate	datetime(3)	NO	-
createdAt	datetime(3)	NO	default CURRENT_TIMESTAMP(3)
updatedAt	datetime(3)	NO	default CURRENT_TIMESTAMP(3)

prescriptions

Field	Type / size	Null	Key / default
id	varchar(36)	NO	PK
medicalRecordId	varchar(36)	NO	Indexed; FK -> medical_records.id
medication	varchar(200)	NO	-
dosage	varchar(100)	NO	-
frequency	varchar(100)	NO	-
duration	varchar(100)	NO	-
instructions	varchar(500)	YES	-

6. REST API Design

Base URL: http://localhost:4000/api/v1. Requests use JSON and protected routes require Authorization: Bearer <token>. The OpenAPI contract documents 32 method/path endpoints.

Resource	Endpoints / methods
Authentication / health	POST /auth/register; POST /auth/login; GET /auth/me; GET /health
Patients (6)	GET, POST /patients; GET, PUT, PATCH, DELETE /patients/{id}
Doctors (6)	GET, POST /doctors; GET, PUT, PATCH, DELETE /doctors/{id}
Appointments (7)	GET, POST /appointments; GET, PUT, PATCH, DELETE /appointments/{id}; PATCH /appointments/{id}/status
Medical records (6)	GET, POST /medical-records; GET, PUT, PATCH, DELETE /medical-records/{id}
Users (3)	GET /users; GET, PATCH /users/{id}

Five-method appointment sequence

Method	Operation	Expected
POST	Create a patient booking	201 Created
GET	Read the created booking	200 OK
PUT	Replace startsAt, reason and notes	200 OK
PATCH	Change selected editable fields	200 OK
DELETE	Remove a pending booking	204 No Content
GET after DELETE	Confirm the resource is absent	404 Not Found

Request examples

```
POST /appointments
{"doctorId":"<doctor UUID>", "startsAt":"<future ISO datetime>",
"reason":"General consultation", "notes":null}

PUT /appointments/{id}
{"startsAt":"<future ISO datetime>",
"reason":"Updated consultation", "notes":null}

PATCH /appointments/{id}
{"reason":"Follow-up consultation"}
```

Success responses use {success:true,data:...}. Errors use {success:false,error:{code,message,...}}. DELETE has no response body. Identifiers and role assignments are server-controlled; PUT requires every editable field, while PATCH preserves omitted fields.

7. Authentication and Security

Local account passwords are hashed with bcrypt before storage. Login verifies the hash and returns a signed HS256 JWT with a one-hour lifetime. Each protected request verifies the token, reloads the active user and applies role and ownership checks. Password hashes are not included in API responses.

Public registration creates only patient accounts. Administrator-provisioned doctor and patient profiles are created transactionally. Account status changes are administrator-only, and the current administrator cannot disable their own account. The web portal stores its token in an HTTP-only same-site cookie.

Zod validates types, lengths, dates and required fields. TypeORM uses parameterized queries. Database credentials, JWT secrets and seed passwords stay in the ignored local .env. The committed SQL export contains structure only.

8. API Testing and Validation

Verification	Recorded result
Postman collection through Newman	44 requests; 0 failed
Postman assertions	77 assertions; 0 failed
Portal / security HTTP checks	18 of 18 passed
Browser inspection	Patient login and portal inspected; Levi/Hayek visible; no console errors at inspection.

Case	Expected behavior
Missing / invalid token	401 Unauthorized
Wrong role / ownership	403 Forbidden
Malformed JSON	400 Bad Request
Invalid DTO or incomplete PUT	422 Unprocessable Entity
Duplicate record or invalid workflow	409 Conflict
Unknown / deleted resource	404 Not Found

The collection verifies full profile and clinical-record CRUD as well as appointment methods. It checks that password hashes are absent and that PATCH preserves omitted fields. Synthetic verification records are cleaned up by the automated runner. The recorded checks cover the tested cases; they do not prove every possible behavior.

Repeat with npm run test:api while the application is running. It invokes pinned Newman tooling through npm and writes a sanitized verification summary. First-time tool download requires internet. After dependencies are installed, the core MySQL/JWT application works locally without cloud authentication.

9. Conclusion

MediConnect implements the required backend stack and a complete patient-to-doctor appointment workflow. Separating controllers, services and repositories makes requests easier to follow and explain. Relational keys keep profiles, bookings and clinical records connected, while authentication and ownership checks restrict access.

The API provides 32 endpoints and supports GET, POST, PUT, PATCH and DELETE. Verification covers the submitted Postman collection, saved database changes and role-specific portal pages. Future work includes deployment-specific rate limiting, controlled schema migrations, audit history and operational safeguards before processing real patient information.

10. References

Course handout: 09_Final_Project_Student_Copy.pdf; ITEC 116 Systems Integration Backend Prototype.

NestJS documentation: https://docs.nestjs.com/security/authentication

TypeORM documentation: https://typeorm.io/docs/data-source/data-source/

Postman documentation:
https://learning.postman.com/docs/tests-and-scripts/running-collections/intro-to-collection-runs/

Implementation: backend/src/; docs/openapi.yaml; docs/database.sql; postman/MediConnect.postman_collection.json; docs/VERIFICATION.json.

11. Submission Notes

Deliverable	Location / status
Source code	https://github.com/Ajerojay/MediConnect
Technical documentation	docs/Final-Project-Documentation.pdf
Database export	docs/database.sql (schema only)
Postman collection	postman/MediConnect.postman_collection.json
Video recording	Student must record and provide the required 5-10 minute video link.
Live presentation	Student demonstrates login, CRUD, code flow, protected routes and database state.

The video link must be added to the cover page when available. This documentation does not claim that the video or presentation has been completed. Keep personal credentials out of the repository and presentation screenshots.